Contents

Specification Review Report

Target: `SPEC.md` — *Model Playground (PyQt5 + uv)*, status v0.1 **Reviewer:** spec-review skill (4-pass method: comprehension → local precision → cross-consistency → implementation simulation) **Grounding:** review cross-checked against the existing `src/model_playground/` implementation and the `tests/` suite (64 passing) so that findings distinguish *spec defects* from *spec code divergence*. **Maturity scale:** 0 = absent · 1 = seriously deficient · 2 = weak · 3 = adequate · 4 = strong · 5 = implementation-grade

1. Executive Summary

`SPEC.md` v0.1 is a **high-quality, near Level 3** specification: it is unusually disciplined for a lab deliverable. It decomposes an `AI Application = Probabilistic Components + Deterministic Systems` system into a clean layering (types → model → metrics → structured → worker → ui), fixes the reliability boundary (validate gate, per-panel fault isolation), makes the metric math formal (TTFT/TPS/cost as equations, guarded against `inf`/`nan` and division-by-zero), and pins down edge cases and failure semantics (E-01…E-15) that most specs of this kind leave implicit. Its invariants I-001…I-012 are each mapped to a test, its constraints K-01…K-04 are measurable, and its traceability matrix (§11) is explicit. The deterministic layer (metrics, structured) is genuinely implementation-grade and the mock-double strategy makes the whole suite runnable offline.

The spec is **not yet Level 3** for one structural reason: **the implementation has already outgrown the document.** Three behaviors live in the code and README but are absent from the spec's normative contracts:

- the reasoning-model **`thinking` channel** (separate from `content`), which affects first-token timing and TPS but is undocumented in `StreamChunk`/`ModelResponse`/C-06/C-07/§5.2;
- the **`OllamaClient` shape** (the spec's single `chat(stream:)` union diverged into an implemented `chat`/`stream_chat` split, and T-17's text is stale);
- the **`structured` and `token_thinking` worker signals** that deliver results to the panels, absent from C-06.

Two **HIGH** findings are pure cross-consistency defects that a verifier would trip on: the E-13 "Ollama unavailable" banner fires on a *reachable-but-empty* daemon (F-003), and I-008 asserts "every `stream` yields a final `finished` chunk" while E-02's raising variant never emits one (F-016).

Strengths (most important).

- Formal, guarded metric definitions (I-004/I-005/I-006/I-007) with exact formulas and unit cases.
- Clear per-panel fault isolation (E-02/E-07/K-02) and a crisp validate gate (I-009 / T-08).
- Testable, measurable constraints; every requirement traces to a contract and a test (§11).
- Strong architecture: a single provider-aware module (I-002/T-02) with the rest seeing only `Model` + `Message` + `GenerationParams`.

Weaknesses (most important).

- **Divergence from the living code** — the `thinking` channel, the `structured`/`token_thinking` signals, and the `chat`/`stream_chat` split are implemented but not specified (F-001, F-002, F-008). Two competent implementers from v0.1 would build materially different systems from the code that already exists.
- **Ambiguous retry and fallback semantics** — the `max_retries` off-by-one (F-009) and the three-state fallback/baseline question (F-003) are the two most likely sources of two-implementation divergence.
- **A few stale cross-references** — §0 cites E-13/E-14/E-15 for "memory/thermal" costs that none of them address (F-014); §11 points E-13/E-15 to "(smoke)" though T-16/T-17 automate them (F-015).

Findings by severity. 0 CRITICAL · 2 HIGH (F-001, F-003) · 8 MEDIUM (F-002, F-004, F-006, F-008, F-009, F-010, F-014, F-016) · 8 LOW (F-005, F-007, F-011, F-013, F-015, F-017, F-018, F-019). Total: 18 findings. (Severities are stated per finding in §4; §20 prioritizes them P0/P1/P2.)

Primary blocker: the `thinking` channel and the `structured`/`token_thinking` signals (F-001, F-008) leave the most exercised behaviors in the suite unspecifiable — a verifier cannot test what is not in the contracts.

Recommendation: resolve the 2 HIGH + 7 MEDIUM items (all P0/P1; see §20) to reach **Level 3**. The fixes are local and additive — contract fields, a couple of sentences of failure semantics, and test-id corrections

— not a redesign. No architectural change is recommended (§17).

---

2. Overall Maturity

Level 3 — Implementation-grade, *pending* the P0/P1 fixes in §20 (presently a strong **Level 2+**).

The specification already clears most of the gate to Level 3: a competent coding agent can implement the deterministic layers (metrics, structured, model interface, mock) with essentially *zero* semantic inference, and those layers carry mechanically checkable invariants (I-001…I-012) with exact unit cases (T-01…T-08, T-14). The probabilistic path (Ollama) is correctly specified as *best-effort*, which appropriately bounds nondeterminism rather than pretending it away.

It does **not** yet reach Level 3 cleanly because of a single class of defect: **the spec drifts from the code that already exists.** The `thinking` channel, the `structured`/`token_thinking` signals, and the `chat`/`stream_chat` split are *observable behaviors in the shipped module and its passing test-suite* that have no counterpart in the normative contracts. Per the skill's core test — *could two competent implementers build materially different systems?* — an implementer working strictly from v0.1 would **not** add a `thinking` field, and a verifier working from v0.1 **could not write** T-17c-as-streaming or a structured-panel test, even though both exist and pass in the code. That is the defining gap between "Level 2 with ambiguities" and "Level 3 with minimal inference."

Two internal *contradictions* (F-003 banner-on-reachable-but-empty; F-016 I-008-vs-E-02) are the more dangerous half: they don't require inventing behavior, they require reconciling behavior the code already exhibits. Both are one-sentence fixes.

After the P0/P1 set (the `thinking`/`structured` channel back-filled into C-01/C-06/C-07, the `chat`/`stream_chat` contract harmonization, the retry-count and fallback-state definitions, and the two contradiction fixes), the document is a **Level 3** spec: minimal semantic inference, objectively testable conformance. A move to **Level 4 (verification-grade)** would additionally require the run-identifier/provenance and the p95 sampling protocol (F-011, F-019) and a normative, exact-match error-string catalog — worthwhile but not required for implementation.

3. Findings Summary

18 findings: **0 CRITICAL · 2 HIGH · 8 MEDIUM · 8 LOW.** They cluster around one theme — *spec drift from the living code* — plus two cross-consistency contradictions and a set of editorial/traceability clean-ups. None is a redesign; every P0/P1 fix is additive or a one-sentence clarification.

ID	Sev	Location	Issue (one line)
F-001	HIGH	C-01, C-03b, C-06, C-07, §5.2	Reasoning-model `thinking` channel is implemented and read-me'd but absent from every contract; it changes TTFT/TPS timing semantics.
F-002	MEDIUM	C-03b, T-17	Spec's single `chat(stream:)` union diverged from the implemented `chat` (→`ModelResponse`) + `stream_chat` (→stream); T-17 text references the wrong method.
F-003	HIGH	E-13, R-16, `registry.discover_registry`	Fallback flag/banner fires on a *reachable-but-empty* daemon, not only on unreachable; "mock fallback" vs "mock baseline" is undefined.
F-004	MEDIUM	§3.4, §8, C-06	Structured mode always uses non-streaming `generate`, but the spec never says the `stream` checkbox is ignored under structured.

ID	Sev	Location	Issue (one line)
F-005	LOW	C-08, §0	UI placeholder `blank = random` is misleading: `MockModel` is seed-deterministic regardless of the seed field.
F-006	MEDIUM	E-15	E-15 requires a panel `warning` on malformed trailing NDJSON that no module emits; behavior is undefined (warn vs silent best-effort).
F-007	LOW	§3.2, C-04	§3.2 lists `COLLECTED`/`VALIDATING` as states, but the surfaced status set / `metrics.status` omits them.
F-008	MEDIUM	C-06, C-07	Worker signals `structured` and `token_thinking` that deliver results to panels are not in C-06; C-07 doesn't say how `ValidationResult` arrives.
F-009	MEDIUM	§3.2, §8, K-03, E-03	`max_retries` off-by-one: "up to N parse attempts" reads as N total, impl runs N+1 (initial + N).
F-010	MEDIUM	§3.1, C-08	Run/Cancel enable is "per state table (§3.1)" but §3.1 gives an ASCII diagram, not an enable/disable rule.
F-011	LOW	K-01, T-11	`p95 < 50ms` names no sample size/population; T-11 services a single task.
F-013	LOW	§9, §11	The T-series skips `T-12` (jumps T-11 → T-13): a dangling slot, no requirement maps to it.
F-014	MEDIUM	§0	§0 cites E-13/E-14/E-15 as the home of "memory/thermal constraints," but none of them addresses resources.
F-016	MEDIUM	I-008, E-02	I-008 "every `stream` yields a final `finished` chunk" contradicts E-02's raising variant which never emits one.
F-015	LOW	§11	Traceability rows point E-13/E-15 to "(smoke)" though T-16/T-17c automate them; "T-10-style" is vague.
F-017	LOW	C-01, C-05	Minor signature drift: `validate(data, schema, raw, …)` and explicit-kwargs `generate` vs the spec's `**params` / `validate(data, schema)`.
F-018	LOW	E-14, T-18	Error-message text is prose with an em-dash; code uses ASCII `--` and embeds the model name in the pull command — exact-match tests would diverge.
F-019	LOW	(new) §14/§3.14	No run identifier / timestamp / version is captured, so a run's metrics are not self-describing after the fact.

Severity discipline. No finding is rated CRITICAL: nothing makes the spec *impossible to implement or fundamentally unverifiable* — the deterministic core is solid and the probabilistic path is honestly bounded. The two HIGH items are *cross-consistency defects that a verifier would catch*, not conceptual gaps. The MEDIUM set is the work that must be done before claiming conformance; the LOW set is improvement/defer.

4. Detailed Findings

Each finding follows the skill's format: **Observation · Why it matters · Potential consequence · Recommended resolution**, with location and severity. "Impl" quotes the existing `src/model_playground/` code that the spec must reconcile with.

F-001 — Reasoning-model `thinking` channel is unspecified

Severity: HIGH **Location:** C-01 (`StreamChunk`, `ModelResponse`), C-03b (`OllamaClient`), C-06 (`RunWorker` signals), C-07 (`ModelPanelView`), §5.2, and the §3.4 timing rule.

Observation `StreamChunk` and `ModelResponse` in C-01 carry only `text`/`usage`. But the implementation and README add a parallel `thinking` channel: `StreamChunk.thinking: str = ""`, `ModelResponse.thinking: str = ""`, a worker `token_thinking` signal, a `_best_effort_usage(text_parts, thinking_parts)` that folds thinking into best-effort token counts, and — critically — the worker counts a thinking delta as the **first token** for TTFT (`if chunk.thinking: if t_first is None: t_first = time.monotonic()`). README calls this out ("a reasoning model's chain-of-thought … is surfaced in its own panel block and counted toward first-token timing"). None of this is in the spec.

Why it matters This is the single highest-priority gap: the behavior is *exercised by the passing suite and documented in the README* yet absent from the contracts. It also changes the **timing semantics** the spec formalizes — §3.4 says TTFT is "to the first non-`done` chunk that carries a non-empty delta," but the code also treats a *thinking-only* non-empty chunk as first-token, which the spec's prose does not cover.

Potential consequence The implementation test fails: an implementer working strictly from v0.1 will not create a `thinking` field, so `MockModel`, the Ollama mapping, the worker, and the panel all lack it. The test test fails: a verifier cannot write a "thinking-only stream still shows progress" or "thinking counts toward TTFT" assertion from v0.1. Two conforming implementations diverge on a headline feature (reasoning models are a central selling point in the README screenshot).

Recommended resolution 1. Add `thinking: str = ""` to `StreamChunk` and `ModelResponse` in C-01, with a note that it is "" for non-thinking models and that a thinking delta counts toward TTFT/TPS. 2. Add `token_thinking(model_id, thinking)` to the C-06 signal set. 3. Add `thinking` to the C-07 `ModelPanelView` and to §5.2 (a separate `(thinking)` block above the answer). 4. Update §3.4 and I-004/I-005 to state TTFT = first non-empty *delta* ***or thinking***, and TPS counts thinking toward `completion_tokens` where the runtime does so (Ollama `eval_count` semantics), with the mock defining its thinking as non-contributing by default. 5. Update C-03b: `OllamaClient` maps the Ollama `thinking` message field to `StreamChunk.thinking` in both `chat` and `stream`, and `_best_effort_usage` folds thinking tokens when a stream ends without a usable `eval_count`. 6. Add T-19 (new): a thinking-only stream surfaces in the panel and affects TTFT; add to §9.3.

F-003 — Fallback state and banner are undefined; banner fires on reachable-but-empty

Severity: HIGH **Location:** R-16, E-13, §0/§1 ("fall back to the built-in `MockModel` registry"), `registry.discover_registry`.

Observation E-13 says: on an *unreachable* daemon the checklist "falls back to the `MockModel` registry" and shows banner `Ollama unavailable - using mock models`. But `discover_registry` returns `used_fallback = len(names) == 0`, i.e. **True** even when the daemon *is* reachable and simply has **zero** locally-pulled models. The mock registry is always built first (`build_default_registry()`), so mocks are a **baseline overlay**, not an exclusive fallback. Neither distinction — *unreachable* vs *reachable-but-empty* vs *reachable-and-populated* — is in the spec, and the banner text ("unavailable") is factually wrong in the reachable-but-empty case.

Why it matters Three distinct runtime conditions are conflated into one boolean with one banner string. The failure test ("what is the caller's observable state?") has two reasonable readings that produce different UIs and different test expectations.

Potential consequence A verifier writing T-16 could assert the banner for *either* "dead port" or "0 models," and they would disagree with a tester who distinguishes them. The reachability question (is Ollama actually running?) is answered incorrectly by the UI in the reachable-but-empty case.

Recommended resolution 1. Define three discovery outcomes: `UNREACHABLE`, `REACHABLE_EMPTY`, `POPULATED`, and a boolean `used_fallback` per the chosen contract. 2. Fix the registry so `used_fallback` reflects *unreachable*, not *empty*; keep an explicit `reachable` flag. 3. Define distinct banner text per outcome (e.g. `Ollama unreachable - mock models only` vs `Ollama online - no local models; mock models available`). 4. State explicitly that the mock variants are a **always-present baseline** overlaid on (not replacing) a populated Ollama list, and that the checklist shows baseline discovered. 5. Update E-13 and add a fourth T-16 sub-case for reachable-but-empty.

F-002 — `OllamaClient` signature diverged: union `chat` vs `chat`/`stream_chat`

Severity: MEDIUM **Location:** C-03b, T-17, `ollama.py`.

Observation C-03b declares one method `chat(model, messages, params, stream: bool) -> "Iterator[StreamChunk] | ModelResponse"`. The implementation splits it into `chat(..., stream=False) -> ModelResponse` (non-streaming, single JSON object) and `stream_chat(...) -> Iterator[StreamChunk]` (NDJSON), and `OllamaModel` calls the two by name. T-17's prose says "`OllamaClient.chat` … parses a multi-line NDJSON stream into `StreamChunks`" — but NDJSON parsing lives in `stream_chat`, not `chat`.

Why it matters A union return type is a conformance hazard (a verifier must test both branches of a polymorphic shape), and the named split is cleaner and is what the code and tests actually use. T-17's cross-reference is stale.

Potential consequence Implementing strictly to C-03b yields one overloaded method and a union return that the existing tests do not call; T-17 as written would target a method that no longer exists by that name.

Recommended resolution Replace the C-03b union with two total methods: `chat(model, messages, params) -> ModelResponse` and `stream_chat(model, messages, params) -> Iterator[StreamChunk]`; state that `OllamaModel.generate` delegates to `chat` and `OllamaModel.stream` to `stream_chat`; fix T-17 to cite `stream_chat` for NDJSON parsing.

F-004 — Structured mode's relationship to the `stream` flag is unspecified

Severity: MEDIUM **Location:** §3.4, §8 (E-03), C-06, `worker._run_structured`.

Observation §3.4 says the interface offers *both* streaming and non-streaming and "the UI picks one." But `_run_structured` always calls the non-streaming `generate()` regardless of the `stream` checkbox, because the full text must be collected before parse/validate. The spec never states that **under structured mode the `stream` checkbox is effectively ignored** (or defines streaming-structured semantics where it would exist).

Why it matters The enable-flags and the meaning of the checkbox set depend on this. A user checking both *Stream* and *Structured* gets non-streaming behavior with no explanation; a tester cannot assert the interaction from the spec.

Potential consequence Two implementers differ: one may try to stream-then-validate (buffering the stream), one may ignore `stream` under structured. Neither reading is wrong from v0.1, so conformance is ambiguous. Also affects TTFT for structured runs (always the E-04 special case `ttft == total`).

Recommended resolution State explicitly: *structured mode collects the full response via the non-streaming path before validation; the `stream` control has no effect while `structured` is set (state this in §3.4, and reflect it in the C-08 enable rules and T-13).* Optionally define buffered-stream-structured semantics as a future variant.

F-008 — `structured` and `token_thinking` worker signals are omitted from C-06

Severity: MEDIUM **Location:** C-06, C-07, §5.2, `worker.py`.

Observation C-06 lists exactly three signals: `token`, `metrics_ready`, `crashed`. The implementation declares five, adding `structured(model_id, result)` (delivers the `ValidationResult` to the panel — the *only* path by which a `VALID`/fail badge appears) and `token_thinking(model_id, thinking)` (see F-001). C-07's `ModelPanelView.structured: ValidationResult | None` never says *how* that value is delivered.

Why it matters The structured-mode UI result — a headline feature (R-09/R-10) — has no specified delivery channel. C-07 references a field whose population mechanism is unspecified.

Potential consequence An implementer who reads only C-06 would not know the panel must subscribe to a `structured` signal; R-09/R-10 conformance (the badge) is not traceable to a contract. The test test fails.

Recommended resolution Add `structured(model_id, ValidationResult)` and `token_thinking(model_id, thinking)` to C-06; in C-07, state that `Metrics_ready` carries the `RunMetrics`, `structured` carries the `ValidationResult`, and `token`/`token_thinking` append to the panel's `text`/`thinking`. Reference both from §5.2.

F-009 — `max_retries` off-by-one (total attempts vs initial+retries)

Severity: MEDIUM **Location:** §3.2, §8/E-03, K-03, T-08(e), `worker._run_structured`.

Observation §3.2 says structured mode "may perform up to `max_retries` parse attempts" and E-03 says "retry up to `max_retries`". K-03 sets `max_retries = 2`. But the implementation loops `for attempt in range(self._max_retries + 1)` — i.e. **one initial attempt plus up to two retries = three generations** — and reports `retries = 2`. "Up to N parse attempts" most naturally reads as **N total**.

Why it matters This is the classic off-by-one that separates two conforming implementations. It directly changes how many generations a structured run performs (and thus total latency and cost) and what `retries` means in `RunMetrics`.

Potential consequence A tester expecting "2 attempts" asserts `retries == 1`; the code asserts `retries == DEFAULT_MAX_RETRIES (2)`. T-08(e) ("after max_retries exhaustion") is satisfiable under both readings, masking the disagreement. Cost-per-task and total latency metrics diverge by a factor of the retry count.

Recommended resolution Adopt one definition and use it everywhere. Recommend: *`max_retries` is the number of **retries after the initial generation**; total generations per panel = `max_retries + 1` (default 2 → 3 generations). `RunMetrics.retries` reports the number of retry generations actually issued.* Reword §3.2 and E-03 accordingly and pin the exact value in T-08 with the explicit count (3 generations for the default, `retries == 2`).

F-016 — I-008 contradicts E-02: "every stream" vs a raising variant

Severity: MEDIUM **Location:** I-008, E-02, `MockModel raising` variant.

Observation I-008 asserts "every `stream` yields a final chunk with `finished=True`." E-02 describes a model that raises *mid-stream* (`MockModel raising` variant raises at `i == 2`), so that `stream` **never** reaches a `finished` chunk.

Why it matters An invariant stated unconditionally is contradicted by an edge case the spec itself defines. The two passages are both normative.

Potential consequence A verifier who reads only I-008 would assert that every mock variant finalizes, and fail against the raising variant; a verifier who reads only E-02 would treat missing-final-chunk as the expected error path. The invariant as written is not universally true.

Recommended resolution Scope I-008 to "every **successfully-completing** `stream` yields exactly one `finished=True` final chunk carrying the usage; a stream that raises mid-stream (E-02) is the defined exception and yields no final chunk." Add the qualifier to I-008 and cross-reference E-02.

F-006 — E-15 "surface a warning on the panel" is unspecified / unimplemented

Severity: MEDIUM **Location:** E-15, `worker`, `ui`.

Observation E-15 requires that on a malformed final NDJSON line the panel keep partial text, use best-effort usage, **and "surface a `warning` on the panel (E-02)."** No module emits a panel warning: `grep` for `warning` across `src/` returns nothing. The worker only substitutes best-effort usage silently and settles as `COMPLETED`.

Why it matters The required observable behavior (a visible warning) has no mechanism described and none implemented; the requirement is untestable as written.

Potential consequence An implementer would either invent a channel or silently omit the warning; a tester cannot assert it. The requirement and the code disagree on an observable UI state.

Recommended resolution Choose one: (a) *drop* the "surface a warning" clause and make E-15 purely "keep partial text + best-effort usage, settle as `COMPLETED`" (simplest, matches the code), or (b) *specify* the channel — e.g. a `RunMetrics.warning: str | None` plus a panel `warning` label — and add a T-17d sub-test. Given the code, recommend (a) and note the optional (b).

F-010 — Run/Cancel enable rules reference a "state table" that is not stated

Severity: MEDIUM **Location:** §3.1, C-08, T-13, T-15.

Observation C-08 says Run/Cancel are "enabled per state table (§3.1)," and T-13 says "control-enable flags match §3.1." But §3.1 contains only an ASCII state *diagram*; it never lists an explicit enable/disable rule. The actual rule lives in `ui._update_running` (Run enabled `not running & valid`; Cancel enabled `running`) but is not in the spec.

Why it matters The control-enable behavior is cited by a test (T-13, T-15) as being defined in §3.1, but §3.1 does not define it. The contradiction test: T-13 cites a table that is not there.

Potential consequence A verifier cannot confirm control-enable behavior against a non-existent table; the test is unverifiable from the spec.

Recommended resolution Add an explicit control-enable table to §3.1: *Run enabled* `state` *= IDLE* *inputs valid (non-empty prompt,* `1` *model, valid seed,* `max_tokens` `1`*) not already RUNNING; Cancel enabled* `state` *= RUNNING; both disabled IDLE.* Wire T-13/T-15 to it. Also clarify the §3.1 diagram's mislabeled `Cancel/Reset from IDLE` edge (cancellation happens on a *running* run).

F-014 — §0 cross-reference: "memory/thermal → E-13/E-14/E-15" is wrong

Severity: MEDIUM **Location:** §0 ("memory/thermal constraints — see E-13/E-14/E-15").

Observation §0 lists the engineering burden of local inference and cites E-13/E-14/E-15 for "memory/thermal constraints." In §8, E-13 = daemon unreachable, E-14 = model not pulled, E-15 = malformed NDJSON. **None** addresses memory/thermal/resource exhaustion.

Why it matters This is a stale cross-reference (an internal-consistency defect): the pointer names edge cases that do not cover the claim it is attached to.

Potential consequence A reader chasing the "resource cost" claim lands on Ollama-availability edge cases, missing the actual resource concerns. Undermines §0's credibility as scope-setting.

Recommended resolution Either (a) add a resource-edge case (e.g. E-16: *inference exceeds host memory/thermal budget — surface a terminal `ERROR/TIMED_OUT` with a resource note; never crash the process, siblings continue*), and cite it, or (b) reword §0 to not claim resource coverage, or (c) soften to "resource constraints are acknowledged but out of scope for v0.1 (documented as a known limitation)." Recommend (b)/(c) for v0.1, optionally add E-16 if resources are in scope.

F-005 — `blank seed = random` is misleading for `MockModel`

Severity: LOW **Location:** C-08 (`seed` placeholder), §0/§10, `MockModel._words`.

Observation The UI placeholder reads "blank = random." For a *real* Ollama model, `seed=None` means the runtime samples freely (plausible). But `MockModel._words` does `seed = params.seed if params.seed is not None else 0` — so a blank seed makes the mock **fully deterministic** (always the `seed=0` sequence). The placeholder over-promises randomness that doesn't happen on the default mock path.

Why it matters A minor but real UI/expectation mismatch; the "random" framing conflicts with R-15's mock-determinism guarantee.

Potential consequence Low: user confusion, not conformance. Could mislead a test expecting seed-varying output from the mock.

Recommended resolution Clarify in C-08/§10 that `blank seed` means "no fixed seed" — nondeterministic *only on runtimes that support it*; `MockModel` is deterministic regardless (blank `seed 0`). Consider per-model placeholder text.

F-007 — `COLLECTED/VALIDATING` states are declared but never surfaced

Severity: LOW **Location:** §3.2, C-04 (`RunMetrics.status`), T-13.

Observation §3.2 defines `COLLECTED` and `VALIDATING` as per-model states, but the surfaced status set (the pill, and `metrics.status`) only ever shows `PENDING/STREAMING/COMPLETED/VALID/ERROR/TIMED_OUT/CANCELLED`. The two are transient/internal and never persisted in a `RunMetrics`.

Why it matters Minor terminology drift between the conceptual state machine and the observable status channel; a tester mapping states to `RunMetrics.status` will find two declared states that can't occur in any metric.

Potential consequence Low: a verifier may look for `COLLECTED`/`VALIDATING` to appear somewhere observable and not find them.

Recommended resolution Annotate §3.2 that `COLLECTED`/`VALIDATING` are **transient internal transitions not surfaced** (only terminal statuses are observable via `RunMetrics`/pill), or fold them into the prose as "concept-only" steps.

F-011 — K-01 `p95 < 50ms` names no sampling population

Severity: LOW **Location:** K-01, T-11.

Observation K-01 requires `p_95 < 50ms` for a posted event-loop task while all models stream, but defines neither the sample size `N` nor the sampling protocol; T-11 services a single posted task and asserts `< 50ms`.

Why it matters A percentile over an undefined population is not measurable (NFR dimension: must be measurable, associated with defined conditions). Single-sample p95.

Potential consequence Low: a tester could "pass" a single fast task and call it p95; conformance is under-specified but not unsafe.

Recommended resolution State the protocol: e.g. "post N=200 tasks during a concurrent run, record each service latency, and assert the 95th-percentile of that sample is `< 50ms`" (the existing `test_ui` already loops 200 — align T-11 to it), or downgrade K-01 to "a representative posted task is serviced within 50ms" and remove the `p95` label to avoid the sampling ambiguity.

F-013 — Test-series skips `T-12`

Severity: LOW **Location:** §9, §11.

Observation The T-series runs T-01…T-11, then T-13…T-18, skipping `T-12`. No requirement maps to T-12 and there is no `(T-12 omitted)` note.

Why it matters A dangling slot in an otherwise fully-traced series invites confusion ("is T-12 missing?") and is a small traceability blemish.

Potential consequence Low: cosmetic, but it weakens the "fully-traced" claim of the traceability matrix.

Recommended resolution Either fill T-12 with the missing coverage it implies (e.g. the E-15 best-effort-usage test, or a CANCELLED-terminal test), or add an explicit note `T-12: reserved/unused`. Recommend reusing T-12 for an E-15 or CANCELLED test.

F-015 — §11 points E-13/E-15 to "(smoke)" though automated tests exist

Severity: LOW **Location:** §11 traceability matrix.

Observation The matrix routes some rows to `(smoke, E-13)` / `(smoke)` for E-13 and E-15, and to `T-10-style` for R-17/E-14 — yet T-16 automates E-13 discovery/fallback, T-17c automates E-15, and T-18 automates R-17/E-14. "T-10-style" is not a real id.

Why it matters The traceability layer (which the spec prides itself on) under-claims its own test coverage and uses a non-id ("T-10-style").

Potential consequence Low: a reader over- or under-counts coverage; the traceability matrix — a stated strength — has stale pointers.

Recommended resolution Correct the rows: `E-13 → T-16`, `E-15 → T-17c`, `R-17/E-14 → T-18`; replace `T-10-style` with `T-18`. Keep §9.5 smoke descriptions as *additional* qualitative checks, not the primary trace.

F-017 — Minor contract signature drift vs implementation

Severity: LOW **Location:** C-01, C-05, C-02, `structured.py`, `model.py`.

Observation Two minor signature drifts: `validate(data, schema, raw="")` in code vs `validate(data, schema)` in C-05; and `generate(self, messages, temperature=…, top_p=…, max_tokens=…, seed=…)` (explicit kwargs) in code vs the spec's `generate(self, messages, **params)`. Both are harmless and arguably cleaner, but they differ from the written contract.

Why it matters Drift between a written contract and the code it claims to specify, even when the code is arguably better, erodes "spec is source of truth" (the document's own stated principle).

Potential consequence Low: a strict implementer would write `validate(data, schema)` and `**params`, diverging from the shipped surface.

Recommended resolution Update C-05 to `validate(data, schema=ANSWER_SCHEMA, raw="")` and C-02 to allow either `**params` (spec form) or the explicit `GenerationParams` fields (code form) — and state that `**params` accepts `GenerationParams` fields, which the implementation may bind explicitly. Add `raw` to the C-05 `ValidationResult` note.

F-018 — Error-message text is prose (em-dash) and differs from code; exact match is fragile

Severity: LOW **Location:** E-14, E-09, T-18, `ollama.py`.

Observation E-14/E-09 give the `model not found` message as prose with an em-dash: `model not found: 'foo' - pull it with ollama pull`. The implementation emits ASCII `--` and embeds the model name in the command: `model not found: '{model}' -- pull it with 'ollama pull {model}'`. The spec document also uses em-dashes throughout (an ASCII-encoding concern for the PDF pipeline).

Why it matters Error strings surface in the UI and are asserted by T-18. Exact-match string tests are brittle; the prose-to-code mismatch (em-dash → `--`, with/without embedded model name) means a strict exact-match test fails.

Potential consequence Low: T-18 must use substring matching (as it likely does) rather than exact equality; otherwise it's fragile.

Recommended resolution Specify error messages as **canonical substrings** the implementation must *contain* (not exact strings): `model not found: '<id>'` and a pull hint containing `ollama pull`. Standardize on ASCII (`--`) per the ASCII-only diagram convention. State that T-18 asserts the substring `model not found:`.

F-019 — No run identifier / timestamp / version is captured (provenance gap)

Severity: LOW **Location:** §3.14 (observability/provenance), C-04 `RunMetrics` (new field), §10.

Observation `RunMetrics` records model_id, status, timings, usage, cost, retries, error — but a run has no **run id, wall-clock timestamp, or app/spec version**. After a run, its metrics are not self-describing: you cannot recover *when*, *which app/version*, or *which run* produced a given metrics record.

Why it matters The provenance test (can an engineer determine what happened and why afterward?) is only partially met: the *per-model* facts are present, but the *run-level* provenance is not. This is the main gap to Level 4 (verification-grade).

Potential consequence Low for a single-invocation demo; material only if runs are logged or compared across time — which the spec lists as a non-goal. So this is a Level-4 stretch, not an implementation blocker.

Recommended resolution Optional (P2): add `run_id: str`, `ts: float` (mono or wall), and `spec_version: str` to `RunMetrics`/the run aggregate, and state these in §3.14. Not required for Level 3.

5. Requirements Review

Are requirements observable? Yes, overwhelmingly so. R-01…R-17 are written to the *observable obligation* form the skill demands: each states a capability, the condition, the input, and the result. The strongest are R-06 (cost from registry prices, not hard-coded), R-09/R-10 (validated object, not prose), R-15 (deterministic mock vs best-effort Ollama — an honest, testable treatment of nondeterminism), and R-16/R-17 (discovery + unpulled-model handling, with a concrete error state). R-08 ties a metric (TTFT) to a structural fact (`TTFT  T_complete`) that is independently checkable (I-004). This is a clear strength.

Are they precise enough? Mostly, with two gaps. R-11 ("apply a retry-then-fallback policy") is precise *in prose* but imprecise *in counts* — it inherits the `max_retries` off-by-one of F-009. R-16 is imprecise about *which* runtime conditions trigger the mock path (F-003): it names "unavailable" but the observable state space has three members. Otherwise the requirements are at Level-3 precision.

Missing requirements. Two gaps: 1. **The `thinking` channel** (F-001) is a first-class, README-documented, test-exercised capability that no `R` mentions — requirements should name it (add `thinking` to R-03/R-08's scope, and note it counts toward TTFT). Without this, the most distinctive feature is requirement-orphans. 2. **Structured-vs-streaming interaction** (F-004) is not covered by any requirement; R-07 (side-by-side) and R-09 (structured) don't state what happens when the user enables both.

Conflicting requirements. One, via the invariant it underlays: R-15 + R-10 coexist fine, but the invariant **I-008 (F-016)** that supports R-05/R-15 conflicts with the **E-02** edge case that supports R-11/E-02 — "every stream finalizes" vs "a model raises mid-stream." That is a requirement-vs-requirement contradiction mediated through the invariant layer, and it is the most serious consistency defect in the requirements set.

Aspirations vs obligations. §1 explicitly separates *intent* (requirements) from *operationalized behavior* (contracts). A few places still read as aspiration rather than obligation — e.g. §0's "pays the corresponding engineering burden" (F-014, which cross-references the wrong edge cases) and K-01's `p95 < 50ms` without a sampling protocol (F-011). These are the only places where "measure this" lacks a measurement definition.

Recommendation (requirements): the requirements are strong and observable; the two HIGH/one MEDIUM items (F-001 thinking-as-requirement, F-003 fallback-state, F-016 invariant-vs-edgecase) are the only ones that must be corrected for conformance. The LOW set is editorial. Score: **4/5** (see §19).

6. Interface and Data-Contract Review

Interface completeness: strong, with three gaps. C-01 (core types) is complete and well-guarded: `Usage` enforces `0` counts (I-001), `GenerationParams.validate()` enforces the ranges declared in C-08 (E-06), `Role.coerce` rejects unknown roles. C-02 (the `Model` ABC) is the cleanest part of the spec: a 3-member interface (`model_id`, `generate`, `stream`) with preconditions (non-empty messages) and postconditions (usage counts) — and the invariance I-002 makes it genuinely substitutable. C-03 (`ModelRegistry`) is complete with the pricing invariant I-003 pinned to it. C-06 (`RunWorker`) is complete *modulo the missing signals* (F-008): `token/metrics_ready/crashed` are declared but `structured/token_thinking` are not, even though C-07's `ModelPanelView.structured` field can only be populated through an undeclared signal.

Schema precision: good, with drift. `GenerationParams` field ranges (C-01) and the C-08 UI constraints agree (`top_p` in `(0,1]`, `max_tokens` in `[1, 100000]`, `temperature` in `[0,2]`, seed blank→`None`). The `ANSWER_SCHEMA` (C-05) is a valid Draft-2020-12 object with `additionalProperties: false` and `minLength/minimum/maximum` guards — well-formed and matches the implementation's `DEFAULT` schema exactly. `parse_json/validate` total-return contracts (never raise) are precise. Two minor drifts: (a) `validate`'s `raw` parameter is in the code but not the C-05 signature (F-017); (b) the spec's `generate(self, messages, **params)` vs the code's explicit kwargs (F-017). Neither is a conformance risk, both are "spec-not-updated" issues.

Input/output ambiguity: present in three places. 1. **`chat` vs `stream_chat`** (F-002): the spec's union return `Iterator[StreamChunk] | ModelResponse` is a polymorphic-branch hazard; the code resolved it into two total methods but the spec didn't. T-17 then cites the wrong method name. 2. **Structured-mode × streaming** (F-004): the interaction is not defined; the `stream` checkbox's effect while `structured` is set is unspecified. An implementer could reasonably buffer-and-validate or ignore `stream`. 3. **Fallback state** (F-003): `discover_registry` returns one boolean, but three runtime conditions map to it; the banner text ("unavailable") is wrong for reachable-but-empty.

Serialization: the only serialization surface is JSON (structured output and Ollama NDJSON). `parse_json` handles the two expected encodings (bare + single "`json fence) precisely, with a clear "never raises, returns`(None, [reason])`'" contract (E-11 / T-14). For Ollama NDJSON, E-15 covers malformed lines with a *best-effort usage* and a *surface a warning* clause — but the warning is neither specified nor implemented (F-006), so E-15 is half a contract.

Compatibility / versioning: not a concern for v0.1 (single version, `spec_version` would be the F-019 provenance add). The `seed` field already anticipates Ollama's `options.seed` support and its absence elsewhere (`best-effort` per R-15).

Overall: interface design is a clear strength; the three gaps (F-002, F-004, F-008) are the contract layer's main work. Score: **3.5/5** (see §19).

7. State and Failure Review

State-machine completeness: strong. §3.1 (run-level `RunState`) and §3.2 (per-model `ModelRunState`) are the spec's best asset. The run is an explicit *aggregate* of per-model states, a single model failure leaves siblings running (E-02/E-07), and "a run is settled only when every panel is terminal" is stated outright. Terminal states are enumerated (`COMPLETED/VALID/ERROR/TIMED_OUT/CANCELLED`), and cancellation is idempotent and total (I-010/E-08/E-12). This is textbook level-3 state work.

Failure semantics: excellent — the crux, and it's handled right. The spec names its two dominant failure modes — *accepting an unvalidated artifact as valid* and *one failure poisoning the others* — and forbids both: I-009 is the validate gate (a panel reaches `VALID` *only* through `validate(...).ok == True`, T-08), and E-02/E-07 isolate faults per panel with the run still settling. Failure detection (raise inside iterator, timeout, HTTP 404, malformed NDJSON, uncaught worker fault) is enumerated E-13..E-15/E-07, and the *caller-observed* consequence of each is stated (panel state + message, siblings continue, no process abort). Retry behavior (E-03) and timeout behavior (K-02/E-02) are present and, for the most part, precise.

Two genuine defects in the state/failure layer (both P0/P1): 1. **F-016 (the most serious):** I-008 asserts "every `stream` yields a final `finished=True` chunk," which E-02's raising variant violates. The invariant must be scoped to *successfully-completing* streams. Until fixed, the invariant layer contains a live contradiction. 2. **F-009 (retry count):** §3.2/§8/K-03/E-03 say "up to `max_retries` parse attempts" but the code runs `max_retries + 1` generations. "State on retry" is ambiguous by one generation — which changes `retries`, total latency, and cost. Define once, cite everywhere.

Two minor state defects (P2): **F-007** — `COLLECTED`/`VALIDATING` are declared as states but never surfaced (they're transient/internal; the observable status set is a subset) — and **F-006** — E-15's "surface a `warning` on the panel" is neither specified (channel/field/label) nor implemented, so it's a half-contract.

Cancellation / partial completion / recovery: cancellation is total and idempotent (I-010), partial text is *preserved* and displayed on the failed panel (K-02/E-02), and there is no "resume" — which the spec correctly implies by making each terminal state fully terminal. The §3.1 diagram's `Cancel/Reset from IDLE` edge is mislabeled (cancellation happens on a *running* run) and is part of F-010's control-enable fix.

Overall: the state and failure story is the spec's strongest dimension — the reason it is near Level 3. With F-016 and F-009 corrected it is a clean, contradiction-free, fully-terminal model. Score: **4/5** (see §19).

8. Determinism and Algorithm Review

Metric algorithms: a clear strength. C-04 defines TTFT, total latency, TPS, and cost as explicit equations with edge-case guards rather than prose:

- **TTFT** = `t_first_token - t_request`, with the non-streaming special case `t_first_token = None` `TTFT = T_complete` (E-04), and the implementation clamps `ttft_s = min(ttft_s, total_s)` so I-004 (`ttft total`) holds *by construction* — not just asserted.
- **TPS** = `completion_tokens / (T_complete - TTFT)` with an explicit zero-interval guard and an `inf`/`nan` guard (I-005), so E-05 (zero completion tokens) yields exactly `0.0`, never `inf`/`nan`. This is the kind of numerical rigor that makes T-04 a reliable acceptance test.
- **Cost** = `N_in/1000·P_in + N_out/1000·P_out` (I-006, exact float), and **cost-per-task** clamps the denominator to 1 (I-007), so an all-failed run is well-defined and division-by-zero is impossible *by construction*.

Each equation is paired with the invariant and the test that pins it. This is the most "implementation-grade" part of the document.

Determinism: honestly bounded. R-15 correctly distinguishes *bitwise* reproducibility for `MockModel` (a `sha256` of `seed|temperature|top_p|max_tokens|prompt` seeds a fixed word sequence — genuinely deterministic, I-012/T-07) from *best-effort* reproducibility for `OllamaModel` (Ollama honors `options.seed`, but float/kernel differences may perturb later tokens, so "token counts and metrics are asserted, bitwise text is not"). This is the mature treatment of a probabilistic component — asserting what *can* be asserted and explicitly disclaiming the rest — and it is directly testable (T-07).

Two determinism/algorithm issues (both P1): 1. **F-001 / thinking → timing.** The implementation counts a `thinking`-only chunk as the *first token* for TTFT, and folds thinking text into the E-15 best-effort token count. Neither is in the spec, so the metric equations as written don't describe the actual first-token and TPS behavior for reasoning models. The equations must be updated to name the `thinking` contribution (or state it explicitly as a documented non-standard extension). 2. **F-009 / retry determinism.** "Up to `max_retries` attempts" is nondeterministic between implementations (F-009); because each retry re-issues *generation* and each generation has cost/latency, the count directly perturbs the very metrics C-04 promises to make reproducible. Fixing F-009 is also a determinism fix.

Ordering / normalization / boundary conditions: ordering of the panel grid is *checklist order* (stated implicitly by the UI code; worth making explicit in §5.1); metric rounding for display is `:.0f` (latency/TTFT ms) and `:.1f` (TPS) / `:.4f` (cost) — display-only, not normative, which is correct. The `seed` blank→`None`→`0`(mock) behavior is a normalization worth stating for the mock (F-005). Boundary conditions (`top_p` open at 0, empty prompt, `max_tokens=0`) are all handled by `validate()` + the UI guards (E-06) and are tested (T-15).

The `**params` freedom test: C-02's `**params` and the code's explicit kwargs are an acceptable implementation freedom (bind the same `GenerationParams` fields), so F-017 is LOW; the *only* thing that matters is that the accepted field set and ranges are fixed, which they are.

Overall: the deterministic layer is implementation-grade and correctly treats the probabilistic one. The two P1 items (F-001 thinking-timing, F-009 retry count) are the only algorithmic work. Score: **4/5** (see §19).

9. Edge-Case Review

§8 (E-01…E-15) is comprehensive and, for an edge-case section, unusually disciplined: each row is a *situation → required behavior* pair, most are tied to a test, and the "failure philosophy" paragraph frames them under two governing principles (no unvalidated-via; no fault-poisoning) that the invariants and E-rows jointly enforce. This is a clear strength against the skill's boundary test.

Coverage is broad. Empty/null/missing input (E-01 empty prompt + zero models, E-05 empty response), malformed input (E-11 code-fence JSON, E-15 malformed NDJSON), maximum/minimum/duplicate/conflicting input (E-06 `max_tokens=0`; E-12 duplicate concurrent run), timeout and unavailable-dependency/partial-failure (E-02 mid-stream failure, E-13 daemon unreachable, E-14 unpulled model, E-15 partial stream failure), and cancellation/resource (E-08 total cancel, K-02 partial-preserved-on-timeout) are all present. The most valuable cases (E-02 fault isolation, E-15 malformed-stream, E-14 unpulled model → `ERROR` not crash) are exactly the ones where an unspecified edge would most likely produce divergent or unsafe implementations.

Two edge-case defects (P1): 1. **F-016 (already flagged):** E-02 (raise-mid-stream) *is* the edge case that contradicts I-008; it's in the edge-case section but not reconciled with the invariant section. The fix is to scope I-008 and cross-reference E-02. E-02 itself is well-specified. 2. **F-006 (E-15's warning clause):** E-15 has three required behaviors — *keep partial text*, *best-effort usage*, *surface a `warning`* — and only the first two are implemented and testable. The `warning` is a dangling edge-case obligation. Either make it a specified, testable behavior (P1) or drop it (P2). As written it's half a contract.

Edge cases that are *rightly* not specified (the freedom test). The spec resists over-specifying: it does not enumerate, e.g., every malformed-message shape, every HTTP status beyond 404, or numeric-overflow

of token counts — because `Usage` guards `0` and the metrics guard `inf`/`nan`, those are handled *invariantly* rather than case-by-case. That is the correct restraint; the skill's "do not assume every conceivable edge case must be specified" is respected.

Missing edge cases worth adding (optional, P2): - **E-16 (resource exhaustion / OOM / thermal)** — §0 mentions "memory/thermal constraints" but cites the wrong E-rows (F-014); a real resource-failure edge case (inference exceeds host budget → terminal `ERROR`/`TIMED_OUT` with a resource note, siblings continue, no process crash) would close that gap. Recommended only if resources are in v0.1 scope; otherwise document as a known limitation. - **CANCELLED terminal with partial metrics** — not covered by an E-row though the worker emits it; a short E-12-adjacent note (cancel produces a `CANCELLED` terminal with whatever partial text existed, partial `usage`) would tidy E-08. Low priority.

Consistency of edge-case behavior: the dominant theme — *one panel's failure never abates the run; the run always settles* — is stated in §3.1 and reinforced in E-02/E-07/E-08/K-02, which is the right kind of internal consistency for edge cases. No two E-rows contradict each other except through the E-02/I-008 mediation (F-016).

Overall: edge-case coverage is a top-quartile of what specs this ambitious usually achieve, with two P1 corrections (F-016 scoping, F-006 E-15 warning) and two optional P2 additions (E-16 resources, CANCELLED-partial note). Score: **4/5** (see §19).

10. Non-Functional Requirement Review

NFRs are concentrated in K-01…K-04 and §7, and §0/§10. They are the spec's weakest dimension relative to its strengths, and the gap to Level 3 lives mostly here. Measurability is the recurring defect: NFRs name a quantity but under-specify the *condition* or *population* under which it is measured.

K-01 (GUI responsiveness, `p95 < 50ms`). Names a measurable target but **no sampling population or protocol** (F-011). The test (T-11) services a single posted task, which is not a p95 over a sample. **Fix:** state `N` and the protocol (the existing `test_ui` already loops 200 — align K-01/T-11 to it), or downgrade to a representative single-task bound. Until then, K-01 is a *performance target*, not a *measurable NFR* by the skill's three-part test (measurable testable defined conditions).

K-02 (failures leave terminal panels; partial text preserved). Fully measurable and testable (T-10). Strength.

K-03 (defaults: `max_retries=2`, `timeout_s=30`). Measurable and testable (`DEFAULT_MAX_RETRIES == 2`). But note F-009: the default is pinned but its *semantics* (total-vs-retries) are ambiguous, so "measurable" "unambiguous." Fix F-009 to make K-03 fully meaningful.

K-04 (offline run/import/test, no provider key). A strong, testable NFR (T-02 import scan + a full offline `pytest` run — 64 passing). One of the spec's best NFRs: measurable, testable, *and* directly demonstrates the I-002 architecture.

Other NFRs (stated, not as K-rows; assess under measurability): - **Reliability / fault isolation** — excellent (I-009, E-02/E-07, per-panel isolation, total cancellation). - **Reproducibility** — excellent and honest (R-15: bitwise for mock, best-effort for Ollama). - **Privacy** — strong by construction: local inference, no cloud, no key, no persistence (stated in §0/§1 non-goals). This is a real, defensible non-functional property. - **Availability / graceful degradation** — excellent (E-13 fallback to mock; the banner is the only wart, see F-003). - **Memory/thermal** — *acknowledged* in §0 but **not** a measurable NFR and cross-referenced wrong (F-014). Either add E-16 with a measurable resource bound, or explicitly declare out-of-scope and remove the cross-reference. - **Observability / provenance** — partial: per-model `RunMetrics` is rich (status, timings, usage, cost, retries, error), but **no run id / timestamp / version** (F-019) and the `warning` channel is missing (F-006). This is the main Level-4 gap. - **Maintainability / testability** — excellent: pure deterministic layers (no Qt/network), a mock double, offscreen Qt, and a fully-traced suite.

Overall: the NFRs that matter most here (offline, fault isolation, reproducibility, graceful degradation, privacy) are specified well; the ones that are under-specified (K-01 sampling, K-03 semantics-via-F-009, K-02-adjacent resources, provenance) are the Level-3/4 work. No NFR is *unsafe* — none is both required and

unenforceable in a way that risks a bad build. Score: **3/5** (measurable-but-sparse population; the two P1 NFR fixes would lift it to 3.5–4). See §19.

11. Security and Trust-Boundary Review

The spec's threat model is the *local developer workstation*, and under that model the security posture is **strong by construction**, not by accident. The skill's caution — "do not impose an unrelated security architecture" — applies: this is a single-user, localhost, no-persistence, no-key desktop tool, so a heavyweight auth/transport-security spec would be wrong here. The spec instead earns its trust properties from its *design choices*, and that is the correct response.

Trust boundaries. One external boundary: the **Ollama daemon** on `localhost:11434`. It is correctly identified as external and not part of the project (§1, E-13), and the app's interaction is a *small localhost HTTP call, not a hosted cloud API* (§0). The `OllamaClient` is the sole provider-aware module (I-002/T-02), so the blast radius of any provider bug is contained to one file.

Secrets / keys. No secrets — a positive finding. No cloud API, no key; `OLLAMA_HOST` is the only environment input (a localhost URL), and it has a sane default. The spec explicitly states "No auth" and "no key" as scope boundaries. Nothing to harden.

Authorization / privilege. Not applicable — single-user GUI, no privileged operations, no multi-tenant surface. The skill's rule "only report omissions that matter given the system's stated scope" applies: requiring auth here would be over-spec.

Input validation. Strong: `GenerationParams.validate()` enforces ranges; `parse_json`/`validate` are total (never raise) and gate untrusted model output behind `jsonschema` (the *core* security concern here is *never accepting a model's unvalidated output as valid* — I-009/T-08). The model output is correctly treated as the untrusted input crossing the reliability boundary.

Unsafe failure modes. Excellent: E-02/E-07 (a fault never aborts the process) and E-13/E-14/E-15 (external failures degrade rather than crash). The dominant *unsafe* behaviors — unvalidated-artifact acceptance and process-aborting faults — are explicitly forbidden. No unsafe failure mode is left open.

External dependencies. `httpx`/`jsonschema`/`PyQt5`/`uv` are pinned in `pyproject.toml` and justified in §10. `httpx` is confined to `OllamaClient` (I-002), a genuine security-relevant containment (a transport bug can't reach the pure layers). The Ollama runtime is a host prerequisite, correctly separated from Python deps.

One minor trust nuance: the E-15 *best-effort* usage and the `OllamaClient` parsing of *arbitrary* NDJSON from the network is a small input-validation surface; E-15 requires it not to crash, which is the right minimum. (A hardening note: validate the `done` line's `eval_count` field's *type* too — the code already wraps this in a `try` and falls back, which is correct.) No new requirement needed.

Recommendation: the security posture for this scope is good (P2 only). The only optional improvement is documenting the Ollama-NDJSON parse as the untrusted-input surface and noting E-15's fall-back is the control. No P0/P1 security finding. Score: **4/5** — strong *for the stated scope*; full "5" would require a stated (even if minimal) input-validation policy for the network-decode path, which is P2.

---

12. Observability and Provenance Review

The provenance test (can an engineer determine *what happened and why* after execution?) is **partially met**: per-model *facts* are richly recorded but *run-level provenance* is absent, and one promised channel (the E-15 warning) is unemitted.

What is captured (strengths). `RunMetrics` (C-04) records `model_id`, `status`, `ttft_ms`, `total_latency_ms`, `tps`, `usage` (with `total_tokens`), `cost_usd`, `retries`, and `error` — a rich, structured per-model record sufficient to explain *any* single model's outcome, including why it failed (`error` string) and whether retries

occurred (`retries`). The UI surfaces status pills, error labels, cost/task, and a `settled/ok` count (`(N ok / M settled)`), which is good *live* observability. The §9.5 smoke eval is a *recorded* (if qualitative) observability artifact. This is the spec's second-strongest dimension after state/failure.

What is missing (F-019, P2, the Level-4 gap). No **run identifier**, **wall-clock timestamp**, or **spec/app version** is captured. After the fact, a `RunMetrics` row cannot answer *when* this ran, *which* app/spec version produced it, or *correlate* a run across panels/logs beyond the in-memory model. For a single-invocation demo this is fine; for any logging/comparison use it is the gap. This is the single change that separates Level 3 from Level 4 — hence P2, not P1.

What is dangling (F-006, P1). E-15's "surface a `warning` on the panel" is a *promised* observability channel that no field, signal, or label implements and no test asserts. Either specify it fully (P1) or drop it (P2). Until then, a malformed-NDJSON partial run is *silent* — the very case where an operator most needs to know something was degraded goes unreported.

Determinism-as-observability (strength). The `seed` field and R-15's reproducibility contract make runs *re-runnable and thus reproducible evidence*, which is a strong, under-appreciated observability property: you can re-derive a deterministic result rather than rely on a captured log.

Recommendation: the *live* observability (pills, errors, cost/task, metrics per panel) is excellent; the gaps are (a) F-019 run-level identifiers (P2, optional, Level-4), and (b) F-006 the unemitted `warning` channel (P1, or P2 if dropped). No P0 observability finding. Score: **3/5** (strong live view, no run-level provenance, one dangling channel). See §19.

13. Testing and Verification Review

Are major requirements testable? Yes — this is the spec's second-strongest dimension. §9 defines Level-3 criteria: the deterministic layers (C-04 metrics, C-05 structured) need **no Qt and no network** (T-01…T-08, T-14), the GUI runs **offscreen** via `QT_QPA_PLATFORM=offscreen` (T-09…T-11, T-13, T-15, E-10), and the Ollama client is **network-stubbed** with `httpx.MockTransport` (T-16/T-17/T-18) — so the *entire* suite is offline and deterministic. The actual suite (64 tests) passes, which is independent evidence the criteria are real, not aspirational.

Are acceptance criteria precise / unambiguous? Mostly, but three criteria inherit upstream ambiguity. - **T-08 / T-13 / T-15** are unambiguous (validate gate, control-enable, empty-input disablements — concrete, assertable). - **T-08(e)** ("after `max_retries` exhaustion the panel is `ERROR`, never `VALID`") is satisfiable under *both* readngs of F-009; it must pin the exact generation count to expose the off-by-one. - **T-11** ("a posted task is serviced `< 50ms`") is a single-sample check, not the `p95` K-01 claims (F-011) — it would pass even if K-01 were false.

Could two testers disagree? In two places: T-11 (single-sample vs p95 sample, F-011) and T-16 (reachable-but-empty vs unreachable banner, F-003). Both are P1, both fixable by pinning the protocol/condition.

Coverage of the case matrix. Positive/negative/boundary/failure/integration/invariant coverage is all represented: positive (T-01/T-05/T-07), negative (T-08 out-of-range/missing/extra-property, T-18 404), boundary (E-05 zero tokens, E-06 max_tokens=0, T-15 empty prompt/zero models), failure (T-10 isolated error, T-17c malformed stream, T-08 retry exhaustion), integration (T-09/T-13 GUI), invariant (T-04 I-004/I-005, T-06 I-007, T-08 I-009). **Notably complete.**

Gaps to fix (P1/P2). (1) F-016: add a test that scopes I-008 to *successful* streams (a raising variant has no final chunk). (2) F-001: add T-19 for the `thinking` channel (thinking-only stream still surfaces and shifts TTFT). (3) F-006: either add a T-17d for the E-15 warning or drop the clause. (4) F-011: align K-01/T-11 to a 200-sample p95 (the loop already exists). (5) F-013: fill/reserve T-12.

Traceability of criteria. §11 maps every R/I/K/E/T id; the test matrix is the spec's proudest artifact. The only weak pointers are the "(smoke)" and "T-10-style" rows (F-015), which under-claim T-16/T-17/T-18. Score: **4/5** — comprehensive and passing; the P1 gaps are *additive* test coverage plus two precision pins, not redesigns.

14. Metrics and Evaluation Review

Metric definitions: a clear strength (the spec's third-strongest dimension). C-04 defines each metric with a formula, a unit, a population, and an edge-case guard, and each metric is tied to an invariant and a test — the skill's "independently reproducible from specified evidence" standard is met for the pure metrics.

Metric	Definition in spec	Guard	Test	Reproducible?
`ttft_ms`	`t_first − t_request`; = `T_complete` when non-streaming (E-04)	`min(ttft, total)` I-004	T-04	
`total_latency_ms`	`t_complete − t_request`	`max(0, …)`	T-04	
`tps`	`completion_tokens / (T_complete − TTFT)`	zero-interval `0.0`; `inf/nan 0.0` (I-005)	T-04	
`cost_usd`	`N_in/1000·P_in + N_out/1000·P_out`	exact float (I-006)	T-05	
`cost_per_success_task`	`Σcost / max(1, #success)`	denom 1 (I-007)	T-06	
`total_tokens`	`prompt + completion`	`0` (I-001)	T-01	

Units / denominator / aggregation are all stated (the `/1000` for per-1k pricing, the `max(1, ·)` denominator, the aggregate `Σcost`). **Edge cases**: zero-interval TPS and division-by-zero cost-per-task are both handled *by construction*, which is the gold standard for a numeric spec.

Interpretation. Metrics are interpretation-ready: the UI displays `latency ms`, `TPS`, `in N / out N`, `cost $`, and a `cost/task` summary with an `(N ok / M settled)` provenance footer. The interpretation is consistent with the math.

"Metrics supplied by the component under test" suspicion. The skill flags metrics *supplied by the thing being measured*. Here `completion_tokens`/`prompt_tokens` are supplied by Ollama (`eval_count`/`prompt_eval_count`) or by the mock tokenizer — i.e. **reported by the model/runtime**, not independently measured. The spec correctly handles this: it treats the runtime counts as the *source of truth for usage* and applies its *own* arithmetic (TPS, cost) to them, while asserting cross-consistency (I-008: streamed count == non-streaming count). This is the right posture — the spec doesn't blindly trust a self-reported metric; it constrains it. The one genuine exposure is **E-15 best-effort usage** (whitespace-count of accumulated text), which is an *approximate* metric; the spec should *label it as approximate* (`cost_usd`/`tps` derived from a best-effort count are also approximate). Recommend a one-line note in C-04/E-15 that a best-effort usage yields *approximate* metrics (a `warning`).

Evaluation (the §15 "comparison" goal). The spec is explicit that comparison is *observational* (metrics + human/JSON inspection), *not* an LLM-as-judge (a non-goal). This correctly resists the temptation to add a subjective "quality score," which would be untestable. Good scope restraint.

Overall. Metrics are implementation-grade and reproducible; the only notes are the F-001 thinking-contribution (which changes TTFT/TPS for reasoning models and must be folded into the definitions) and the E-15 *approximate*-metric labeling (P2). No P0/P1 metric defect. Score: **4.5/5** — the strongest individual dimension; the P1 thinking update is additive. See §19.

15. Traceability Review

§11 is the spec's proudest artifact: an id→contract→test matrix that is unusual in a document this size. The intent→requirement→contract→invariant→test→evidence chain is largely intact and *machine-checkable* in places (T-02 literally scans source for I-002; T-08 literally asserts I-009).

What traces cleanly. Most every R and I resolves to a concrete C and a T: `R-01→C-02→T-02`, `R-05→C-01/I-001→T-01`, `R-06→C-03/I-003→T-03/T-05`, `R-15→C-01/MockModel→T-07`, `I-008→C-03b→T-07`, `I-010→C-06→T-09`, `I-011→§3.3→T-11`, `E-02→§3.2→T-10`. The two-way links (id→trace in §11 *and* "Verified by" in §6) are consistent. This is the level of traceability that earns the "Level 3" claim.

Broken / weak links (P2, the F-015 set). Three rows point to *prose* where an automated test exists: 1. `R-16/E-13 → (smoke, E-13)` — but **T-16** automates discovery+fallback (dead-port fixture, no real network). 2. `E-15 → (smoke)` — but **T-17c** automates malformed-stream handling. 3. `R-17/E-14 → T-10-style` — but **T-18** automates the 404→`ERROR` path; "T-10-style" is not a real id.

These don't break conformance; they *under-state coverage* and weaken the very claim (fully-traced) that makes §11 valuable.

The one genuine broken link (P1): F-016 is a traceability-adjacent *consistency* defect — I-008 traces to T-07, but T-07 asserts the "successful stream finalizes" property while E-02 traces to T-10 and asserts the *opposite* for the raising variant. The id-level trace is fine; the *semantics* the ids anchor contradict. Fix is in §16.

Missing links. The `thinking` channel (F-001) has no R/I/T — once added it needs R-03/R-08 scope + a T-19. The `structured`/`token_thinking` signals (F-008) trace through C-06/C-07 but to no test; add the T coverage. `T-12` is a dangling slot (F-013).

Recommendation: traceability is strong; the fixes are mechanical (point rows to T-16/T-17c/T-18, fill T-12 or mark reserved, add T-19 for thinking). No structural change. Score: **3.5/5** (the matrix is complete in spirit; the stale pointers and the F-016 semantic contradiction keep it from 4+). See §19.

---

16. Internal-Consistency Review

The skill's cross-check: *do later sections contradict earlier ones? Does terminology stay stable?* The spec is **mostly consistent**, but has **two genuine contradictions** and a small pile of stale cross-references — the most serious defects in the whole report are here.

Contradiction 1 (F-016, MEDIUM): I-008 E-02. I-008 ("every `stream` yields a final `finished=True` chunk") contradicts E-02 (a model can raise *mid-stream*, so no final chunk). Both are normative. **This is the single most important consistency fix in the report.** Resolution: scope I-008 to *successfully-completing* streams and cross-reference E-02; keep T-07's assertion on successful streams only.

Contradiction 2 (F-003, HIGH): §0/§1 "fall back to mock" E-13 banner `discover_registry` return. The three statements disagree on *which* conditions trigger the mock path and on what `used_fallback`/the banner mean. `discover_registry` returns `used_fallback = len(names)==0`, so the "unavailable" banner shows for a *reachable-but-empty* daemon — factually wrong. Resolution: define three outcomes (unreachable / reachable-empty / populated), fix the return and banner text, and clarify baseline-vs-fallback.

Terminology consistency. Terms are *mostly* stable and well-chosen: `RunState`/`ModelRunState`, `RunMetrics`, `ValidationResult`, `StreamChunk`, `ModelResponse` are used consistently. One overload: **"fallback"** is used two ways — *exclusive* ("fall back to mock") and *baseline-overlay* (mocks always present) — this is the root of F-003, and the fix is to pick one meaning and state it (recommend: mocks are an *always-present baseline* overlaid on discovered Ollama models). One minor: `COLLECTED`/`VALIDATING` declared-but-unsurfaced (F-007, LOW).

Stale cross-references (the F-015/F-014 set + a few). §0 cites E-13/E-14/E-15 for "memory/thermal" which none addresses (F-014); §11 routes E-13/E-15/R-17 to "/smoke/T-10-style" though T-16/T-17c/T-18 exist (F-015); `chat` vs `stream_chat` is named inconsistently between C-03b and T-17 (F-002). These are all *editorial-after-the-fact* defects — the prose was written, then the code/tests evolved — and all are mechanical fixes.

Filenames / command names / defaults. Consistent: `model-playground` script matches `app.py`; `max_retries=2/timeout_s=30` are consistent across K-03 and code (modulo F-009 semantics); `ANSWER_SCHEMA` matches code. The `seed: int | None = None` is consistent across C-01/C-08 and the UI (blank→None).

No later-section silent override. No later section quietly redefines an earlier term's meaning (the F-003 fallback is a *genuine* contradiction, not a silent later-override — good, that it's *visible* is why C-03b's "source of truth" claim doesn't collapse).

Recommendation: fix the two contradictions first (F-016 P1, F-003 P0), then the stale references (F-015, F-014 P2), and resolve the "fallback" overload (root of F-003). After that the document is internally consistent. Score: **3/5** — two real contradictions, but both are small, well-located, and have clear resolutions. See §19.

17. Architecture Review

Does the architecture support the requirements? Yes — decisively. The skill's rule "do not redesign merely because another architecture is preferable" is fully respected: the proposed architecture is *correct* for this problem, and no finding recommends a redesign. The layering `types → model(ABC) → {OllamaModel, MockModel}  OllamaClient → metrics / structured → worker → ui` maps cleanly to §0's "`AI Application = Probabilistic Components + Deterministic Systems`": the *deterministic* boundary (types, metrics, structured, registry, worker) is fully specified and pure, the *probabilistic* component (`OllamaModel`/`MockModel`) sits behind the `Model` ABC, and the two touch only through `Model` + `Message` + `GenerationParams`.

Component responsibilities. Clean and single-responsibility: `OllamaClient` is the *only* provider-aware module (I-002/T-02), pricing lives only in `ModelRegistry` (I-003/T-03), metrics/structured are pure and headless (no Qt/network), the worker owns one model and emits only via queued signals, and the UI does *zero* inference (R-12/I-011). Each module's job fits its file; this is a strong separation.

Dependency direction. Correct and acyclic: the pure layers (metrics, structured, types) depend on nothing but stdlib + jsonschema; the UI/worker depend down; `OllamaClient` is the only thing that touches `httpx`; `OllamaModel` lazily imports the client so `httpx` stays out of the pure path (I-002). The T-02 import scan *enforces* this direction automatically — an unusual and excellent architectural guarantee.

State ownership. Clear: the registry owns model+pricing; each worker owns its model and its run; the UI owns the aggregate view and the panel map. No shared-mutable-buffer-across-threads except via queued signals (§3.3, I-011) — the standard correct pattern for a Qt worker.

Failure boundaries. The *most important* architectural property — *a single model's failure cannot poison the others or abort the process* (E-02/E-07/K-02/I-010) — is *architecturally enforced*, not just a runtime check: per-panel workers with per-panel terminal states, a cancel-closed-down contract, and a settle-on-all-terminal aggregate. This is the architecture doing heavy lifting for the reliability story.

One architecture-consistency note (not a redesign). `discover_registry` returns mocks *first* (baseline overlay), which the UI uses as the always-present set + discovered Ollama models — but §0/§1/E-13 read this as *exclusive* fallback. That's F-003, and it's a *semantics* mismatch, not an architecture mismatch: the baseline-overlay model is actually the *better* design (offline capability is always present). The fix is to fix the prose (§0/§1/E-13) to match the (good) code, or to make the registry exclusive — recommend the former.

Recommendation: the architecture is *correct and should not change.* All findings are within-architecture (a missing field, a missing signal, two contradictions, stale refs). No P0 architecture change. This is a rare strength — the spec's architecture is the model answer to "deterministic boundary around a probabilistic

component." Score: **4.5/5** (correct layering, enforced dependency direction, architecturally-guaranteed fault isolation). See §19.

---

18. Implementation-Agent Readiness

Question: could a strong coding agent implement this spec *without* asking material semantic questions?

Answer: YES — WITH MINOR CLARIFICATIONS (currently **NO — the P0/P1 items are blocking** for a *faithful* implementation, because a faithful implementation is exactly the *shipped* one with `thinking`/`structured`/`stream_chat` — which the spec doesn't yet describe). The distinction is important: the spec is *implementable* (an agent can build a working app from it), but it is not yet *faithful* — a second agent building strictly from v0.1 would produce a *different* app than the one that exists and passes, which is the Level-2→Level-3 gap.

Minimum blocking questions (must answer before claiming v0.1 is the source of truth): 1. **(F-001, P0)** Is the reasoning-model `thinking` channel in scope? It is *implemented and tested* but *unspecified*. Resolution direction: **yes** (back-fill C-01/C-03b/C-06/C-07/§5.2 + add T-19), because the shipped behavior and README demand it. 2. **(F-003, P0)** What are the three discovery outcomes and what does `used_fallback`/the banner mean? Are mocks a *baseline* or an *exclusive fallback*? Resolution direction: **baseline-overlay, three outcomes, distinct banners** (matches code, better design). 3. **(F-016, P1)** Is I-008 scoped to *successful* streams? Resolution direction: **yes** (cross-ref E-02), one-line fix. 4. **(F-009, P1)** Is `max_retries` the *retries-after-initial* (total = N+1 = 3) or *total attempts* (N)? Resolution direction: **retries-after-initial** (matches code; pin T-08 to 3 generations). 5. **(F-008, P1)** Are the `structured` and `token_thinking` signals part of C-06? Resolution direction: **yes** (they're in the code; required for R-09/R-10 conformance). 6. **(F-002, P1)** Is `OllamaClient` `chat`/`stream_chat` or one `chat(...,stream=)`? Resolution direction: **two methods** (matches code/tests; fix T-17).

Non-blocking questions (reasonable choice, no material impact). F-004 direction of structured×streaming (recommend: structured ignores stream via non-streaming collect), F-005 blank-seed placeholder wording, F-006 E-15 warning (drop or implement), F-007 COLLECTED/VALIDATING surfacing, F-011 p95 sample protocol, F-013 T-12 fill/resolve, F-014 resource NFR (in or out of scope), F-015 stale trace pointers, F-016 signature drift, F-018 error-string exact match, F-019 run-level provenance fields. Each has a recommended direction; none blocks a working build, only conformance precision.

Readiness verdict by layer. *Deterministic layers* (metrics, structured, model, types, registry): **READY** — implementation-grade, zero inference. *Probabilistic path* (ollama client, thinking): **READY WITH MINOR FIXES** (F-001/F-002 back-fill). *GUI/worker/state* : **READY WITH MINOR FIXES** (F-008/F-010). *Non-functional* (K-01/K-03): **READY WITH MINOR FIXES** (F-011/F-009). *Everything else* : **READY**.

Overall readiness: READY WITH MINOR CLARIFICATIONS → after the P0/P1 set, **READY** for a faithful, conforming implementation. The blocking item is not *conceptual* — it's *specification-drift*: get the spec to describe what the code already does and the spec becomes verification-grade-for-implementation (Level 3). Score: **3.5/5** (implementation-grade for the deterministic 80%; the P0/P1 set closes the gap to the probabilistic + UI 20%). See §19.

19. Quality Scorecard

Dimension	Score	Note
Scope clarity	4	§0 intent + explicit non-goals; F-014 mis-citation is the only wart

Dimension	Score	Note
Terminology	3.5	one overload ("fallback," F-003); `COLLECTED/VALIDATING` unsurfaced (F-007)
Requirement precision	4	observable & mostly fixed; F-009/F-003/F-001 precision gaps
Interface completeness	3.5	`Model/Registry` clean; F-002/F-004/F-008 gaps
Data-contract completeness	4	C-01 guarded; F-001 `thinking` + F-017 drift
State/lifecycle definition	4	aggregate terminal model; F-016 scope fix
Algorithm precision	4	guarded equations; F-001 thinking→metrics
Failure semantics	4	excellent; F-016/F-006/F-009 corrections
Edge-case coverage	4	E-01…E-15 broad; F-016/F-006, optional E-16
Non-functional requirements	3	K-04/K-02 strong; K-01 sampling, K-03 semantics weak
Security specification	4	strong *for scope*; P2 only
Observability/provenance	3	rich per-model; no run-level (F-019), F-006 dangling
Testability	4	fully offline/offscreen suite; F-011 sample, T-12 gap
Evaluation/metrics	4.5	strongest dimension; reproducible, guarded, honest
Traceability	3.5	§11 model matrix; stale pointers F-015, T-12 gap F-013
Internal consistency	3	two contradictions (F-016/F-003), small & well-located
Architecture consistency	4.5	model answer; enforce direction, fault isolation
Implementation readiness	3.5	deterministic 80% ready; P0/P1 closes last 20%

Weighted read. Four dimensions score 4.5; ten score 4; the three 3-score dimensions (NFRs, observability, internal consistency) are where the *remaining work* and the *most serious defects* live. There are no 0/1/2 scores anywhere — the spec is uniformly competent; the gap to 5 is precision-and-drift, not missing structure.

---

20. Remediation Plan

Findings grouped by priority. **No redesign is recommended** (§17). Each P0/P1 is additive or a one-line clarification; P2 is deferrable. Implementation notes point to the `src/` location that the spec must be reconciled with (or created for), since the shipped code is the implementation the spec must match.

P0 — Blocking (resolve before claiming v0.1 is the source of truth)

- **F-001 (HIGH) — Back-fill the `thinking` channel.** Add `thinking: str = ""` to `StreamChunk` & `ModelResponse` (C-01); add `token_thinking` to C-06; add `thinking` to C-07/§5.2 (§5.2: a `(thinking)` block above the answer); update §3.4 & I-004/I-005 so TTFT/TPS account for a thinking delta; add `OllamaClient` mapping of the Ollama `thinking` field + best-effort folding; add **T-19** (§9.3: a thinking-only stream surfaces and shifts TTFT). *Code already does this (`types.py`, `ollama.py`, `worker.py`, `ui.py`) — the work is to* write it.
- **F-003 (HIGH) — Define discovery outcomes & fallback semantics.** Define three outcomes (`UNREACHABLE` / `REACHABLE_EMPTY` / `POPULATED`), fix `used_fallback` to mean *unreachable* (not *empty*), give each a distinct banner, and state that mocks are an **always-present baseline** overlaid on discovered models. Amend §0/§1/E-13; add a T-16 reachable-empty sub-case; add an E-row or note for the banner per outcome. *Code: `registry.discover_registry` return + banner in `ui._refresh_banner`.*

P1 — Important (resolve before claiming conformance / Level 3)

- **F-016 (MEDIUM) — Scope I-008 to successful streams.** One line: "I-008 applies to *successfully-completing* `streams`; a mid-stream raise (E-02) is the defined exception." Cross-ref E-02. Add a T-07/T-10 assertion for the raising-variant.*
- **F-009 (MEDIUM) — Pin `max_retries` semantics.** Adopt *retries-after-initial* (total = `max_retries + 1` = 3); reword §3.2/E-03; pin T-08(e) to 3 generations & `retries==2`. *Code already `range(max_retries+1)`.*
- **F-008 (MEDIUM) — Add the missing signals to C-06.** `structured(model_id, ValidationResult)` and `token_thinking(model_id, thinking)`; state in C-07 how each field of `ModelPanelView` is populated. *Code already emits them.*
- **F-002 (MEDIUM) — Harmonize `OllamaClient` to `chat`/`stream_chat`.** Replace the union return with two total methods matching the code; fix T-17 to cite `stream_chat` for NDJSON. *Code already split.*
- **F-010 (MEDIUM) — Add the control-enable table to §3.1.** Run enabled IDLE valid 1 model not RUNNING; Cancel enabled RUNNING. Wire T-13/T-15; fix the mislabeled `Cancel from IDLE` edge. *Code: `ui._update_running`.*
- **F-006 (MEDIUM) — Resolve E-15's unemitted `warning`.** Recommend **drop** the "surface a `warning`" clause (matches the silent-best-effort code) OR fully specify a warning channel + T-17d. Label E-15 best-effort metrics as *approximate*.

P2 — Improvement (safe to defer / Level-4 stretch)

- **F-005 (LOW)** — Clarify the `seed` blank placeholder: *"no fixed seed; nondeterministic only on runtimes that support it; mock is deterministic"*.
- **F-007 (LOW)** — Annotate `COLLECTED`/`VALIDATING` as transient/unsurfaced internal states.
- **F-011 (LOW)** — Define K-01's sample population/protocol (align to T-11's 200-task loop) or downgrade the `p95` label.
- **F-013 (LOW)** — Fill T-12 (recommend: E-15 best-effort or a CANCELLED-terminal test) or mark it `reserved`.
- **F-014 (MEDIUM)** — Fix §0's `E-13/E-14/E-15 → memory/thermal` mis-citation: either add **E-16 resource-exhaustion** (terminal `ERROR`/`TIMED_OUT`, siblings continue, no process crash) and cite it, or declare resources out-of-scope and remove the cross-reference.
- **F-015 (LOW)** — Correct §11 stale pointers: `E-13 → T-16`, `E-15 → T-17c`, `R-17/E-14 → T-18` (drop "T-10-style"), keep §9.5 as *supplementary* qualitative checks.
- **F-017 (LOW)** — Align C-01/C-02/C-05 to the code: `validate(data, schema=ANSWER_SCHEMA, raw="")`; `generate(..., **params)` accepting `GenerationParams` fields.
- **F-018 (LOW)** — Specify error messages as required **substrings** (canonical `model not found: '<id>'` + an `ollama pull` hint), ASCII `--`; pin T-18 to substring match.
- **F-019 (LOW)** — Optional Level-4: add `run_id`, `ts`, `spec_version` to the run aggregate / `RunMetrics` for run-level provenance (§3.14). Not required for Level 3.

Priority summary

Priority	Findings	Work
P0	F-001, F-003	Back-fill `thinking`; define 3 discovery outcomes + banner
P1	F-016, F-009, F-008, F-002, F-010, F-006	Scope I-008; pin retries; add signals; split client; enable table; E-15 warning
P2	F-005, F-007, F-011, F-013, F-014, F-015, F-017, F-018, F-019	Editorial / precision / Level-4 stretch

Key insight. The P0/P1 set is *almost entirely spec-writing, not engineering* — the code already realizes the intended behavior; the spec simply lags the implementation. Applying P0+P1 brings v0.1 to **Level 3** (faithful, verifiable implementation-grade).

---

21. Final Verdict

```
Specification maturity:
Level 3 - Implementation-grade (pending P0/P1; currently a strong Level 2+)

Implementation readiness:
READY WITH MINOR CLARIFICATIONS → READY after P0+P1 (6 findings; all additive/one-line).

Primary blocker:
F-001 + F-003 - the reasoning-model `thinking` channel and the three-state
        discovery/fallback semantics are the highest-value behaviors exercised by
        the suite yet absent from the normative contracts, so a faithful
        implementation-and-verification pair cannot be derived from v0.1.

Most important improvement:
Correct the two genuine contradictions (F-016 I-008 E-02; F-003 fallback
        semantics) and back-fill the spec to match the code that already
        implements `thinking`/`structured`/`stream_chat` - that single act of
        specification-drift correction is what moves the document from Level 2
        to Level 3 without any redesign.
```

Strengths (recap). (1) Exceptional deterministic core: guarded, formula-based, reproducible metrics (Evaluation/metrics 4.5). (2) Architecturally-enforced fault isolation and a single provider-aware module with an *automatic* import-scan test (Architecture 4.5, Security 4). (3) Comprehensive edge-case and failure coverage with a coherent governing philosophy (Edge-case 4, Failure 4). (4) An unusual, fully-traced id matrix (§11) that ties intent→requirement→contract→invariant→test→evidence.

Weaknesses (recap). (1) *Specification drift* — the spec lags the code on the headline (`thinking`) and structural (`structured`/`token_thinking` signals, `chat`/`stream_chat` split) features. (2) Two genuine *contradictions* (I-008 E-02; fallback semantics). (3) Under-measured NFRs (K-01 sampling, K-03 semantics-via-F-009) and the Level-4 *provenance* gap (F-019). All are local and additive; none is a redesign.

Report generated by the spec-review skill (4-pass method). All 18 findings are reproducible from `SPEC.md` and the `src/` implementation in `labs/week1/chapter1`. Recommended path: apply P0+P1 to reach Level 3; P2 is a Level-4 stretch.